Las puertas cuánticas son matrices unitarias

Genesis Silesky

20 de octubre de 2026

1. Las puertas cuánticas son matrices unitarias

1.1. Objetivo

Ver que aplicar una puerta equivale a multiplicar una matriz por el vector de estado, y por qué esa matriz debe ser unitaria.

1.2. Conceptos previos

Concepto previo: multiplicación matriz–vector y unitariedad

Ver sección **??**: matrices unitarias ($U^\dagger U = I$) y notación de Dirac. Recordatorio: el producto $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)\left(\begin{smallmatrix} x \\ y \end{smallmatrix}\right) = \left(\begin{smallmatrix} ax+by \\ cx+dy \end{smallmatrix}\right)$.

1.3. Fundamento teórico

Una puerta de un qubit es una matriz $U \in \mathbb{C}^{2\times 2}$ unitaria y el estado evoluciona como $|\psi'\rangle = U|\psi\rangle$. Las más usadas:

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{1}$$

Con $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) =: |+\rangle$ y $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) =: |-\rangle$. *(Aquí se usa la notación $|\pm\rangle$ solo como abreviatura.)*

Por qué unitarias. Si $U^\dagger U = I$ entonces $\|U|\psi\rangle\|^2 = \langle\psi|U^\dagger U|\psi\rangle = \||\psi\rangle\|^2$: la normalización (probabilidades que suman 1) se conserva. Además U es invertible con $U^{-1} = U^\dagger$, es decir, la evolución cuántica es *reversible*. Para H se cumple $H^\dagger = H$ y $H^2 = I$.

1.4. Código (completamente comentado)

```
# %% [markdown]
# # 02 - Las puertas cuanticas son matrices unitarias
# H = 1/sqrt(2) * [[1, 1], [1, -1]]

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit  # un circuito es una lista de puertas
from qiskit.quantum_info import Operator, Statevector
# Operator convierte un circuito (o una puerta) en su matriz unitaria.

# %% Celda 2: la matriz de cada puerta
# Para cada puerta construimos un circuito de 1 qubit con solo esa puerta
```

```python
# y le pedimos a Qiskit su matriz.
for nombre, puerta in [("H", "h"), ("X", "x"), ("Z", "z")]:
    qc = QuantumCircuit(1)
    # getattr(qc, "h")(0) es lo mismo que qc.h(0): aplica la puerta al qubit 0.
    getattr(qc, puerta)(0)
    # np.round limpia el ruido numerico (0.7071067811865476 -> 0.707).
    print(nombre, "=\n", np.round(Operator(qc).data, 3))
# H: crea superposicion. X: "NOT" cuantico (intercambia |0> y |1>).
# Z: deja |0> igual y cambia el signo de |1> (puerta de fase).

# %% Celda 3: aplicar una puerta = multiplicar matriz por vector
H = Operator.from_label("H").data  # matriz 2x2 de Hadamard
ket0 = np.array([1, 0])            # el vector |0>
# El operador @ es el producto matricial de numpy.
print("H @ |0> =", H @ ket0)       # (1/sqrt2, 1/sqrt2): superposicion igual

# Qiskit hace exactamente lo mismo con .evolve(): aplica el operador al estado.
evolucionado = Statevector.from_label("0").evolve(Operator.from_label("H"))
print("Statevector:", evolucionado.data)

# %% Celda 4: las puertas son unitarias
# Una matriz U es unitaria si U^dagger U = I, donde U^dagger es la
# transpuesta conjugada (.conj().T). Esto garantiza que la norma del
# vector se conserve: una puerta nunca rompe la normalizacion.
print("H^dagger H = I:", np.allclose(H.conj().T @ H, np.eye(2)))

# Como consecuencia, toda puerta es reversible: su inversa es U^dagger.
# En particular H es su propia inversa (H H = I), lo que se usa en 05a.
print("H H = I:", np.allclose(H @ H, np.eye(2)))
```

1.5. Explicación celda por celda

Celda 1 `Operator` convierte un circuito en su matriz.

Celda 2 Para cada puerta crea un circuito de un qubit y extrae su matriz. `getattr(qc, "h")(0)` equivale a `qc.h(0)`; se usa para recorrer varias puertas en un ciclo.

Celda 3 Calcula $H|0\rangle$ con `@` (producto matricial de NumPy) y con `.evolve()` de Qiskit; ambos dan $(1/\sqrt{2},\ 1/\sqrt{2}) \approx (0.707,\ 0.707)$.

Celda 4 Verifica numéricamente $H^\dagger H = I$ y $H^2 = I$. `np.allclose` compara con tolerancia numérica (no se usa `==` con decimales).

1.6. Salida obtenida

```
H =
 [[ 0.707+0.j  0.707+0.j]
 [ 0.707+0.j -0.707+0.j]]
X =
 [[0.+0.j 1.+0.j]
 [1.+0.j 0.+0.j]]
Z =
 [[ 1.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j]]
H @ |0> = [0.70710678+0.j 0.70710678+0.j]
Statevector: [0.70710678+0.j 0.70710678+0.j]
H^dagger H = I: True
H H = I: True
```

1.7. Interpretación

Cada puerta es una matriz concreta y calculable. El resultado $H|0\rangle = (0.707, 0.707)$ confirma una superposición con $P(0) = P(1) = \frac{1}{2}$.

1.8. Ejercicios

1. Calcule a mano $XZ|0\rangle$ y $ZX|0\rangle$. ¿Conmutan X y Z?
2. Verifique en el código que X y Z son unitarias.
3. Calcule $H|1\rangle$ y compruebe que sus probabilidades son $\frac{1}{2}, \frac{1}{2}$ aunque una amplitud sea negativa.

1.9. Errores comunes

- Con varios qubits, la matriz de un circuito usa el orden little-endian de Qiskit; para un solo qubit no hay ambigüedad.
- Comparar matrices con `==`: use `np.allclose`.